

Multi-Tasking and Multi-Threading

1.What is a Program?

A **program** is a set of **instructions** written in a computer language that tells the computer **what to do**.

It is stored on disk (like .py, .c, .exe files) and runs when the user or system starts it.

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Hello World");
```

```
    return 0;
```

```
}
```

2.What is a Process?

A **process** is a **program that is currently running in the computer's memory**.

Program under execution is also called Process.

When you start a program, the OS gives it CPU time, memory, and other resources – this running instance is called a **process**.

You open **Chrome** → Chrome becomes a **process**.

3.What is a Thread?

A **thread** is the **smallest unit of execution** inside a process.

A process can have **one thread** or **many threads**, and each thread can run a different part of the program at the same time.

Threads **share the same memory** of the process but run independently.

Example

Imagine you open a **web browser** (like Chrome):

- The **browser process** starts.
- Inside that process:
 - One thread loads the webpage

- One thread downloads images
- One thread plays a video
- One thread handles user input

All these threads run **together**, making the browser fast and smooth.

Feature	Multitasking	Multithreading
Meaning	Running multiple programs at the same time.	Running multiple threads inside one program at the same time.
Unit of work	Process (program).	Thread (small part of a program).
Memory	Each process has separate memory .	Threads share the same memory of the process.
Speed	Slightly slower because switching between processes takes time.	Faster because threads are lightweight.
Use-case	When the user wants to run many apps together.	When one app must do many tasks together.
Example 1	Playing music + typing in Word + downloading a file.	YouTube: video playing + comments loading + downloading + suggestions loading.
Example 2	Browsing + running VS Code + watching a movie.	Chrome: one tab loading page, another rendering images, another handling input.
Used in	Operating systems (Windows, Mac, Linux).	Apps like browsers, video players, servers.

4.Thread Scheduling

Thread scheduling means:

- Many threads want to run.
- OS decides **which thread runs first**, based on **priority**.
- Each thread gets a small time to run (called a **time slice**).
- After its time slice is over, the CPU moves to another thread.

Even if threads belong to the **same program**, the **operating system** controls their timing.

Easy Example

Imagine YouTube app has 4 threads:

1. **Thread A** – Play video
2. **Thread B** – Play audio
3. **Thread C** – Load comments
4. **Thread D** – Download thumbnail images

Priority

- Video thread → priority 1 (highest)
- Audio thread → priority 2
- Comments loading → priority 3
- Thumbnail downloading → priority 4 (lowest)

Feature	Thread Context Switching	Process Context Switching
Meaning	Switching CPU from one thread to another within the same process .	Switching CPU from one process to another different process .
Memory	Threads share the same memory , so switching is light and fast.	Each process has separate memory , so switching takes more time.
Cost	Low cost (fast).	High cost (slow).
What changes?	Only the thread's registers, stack pointer , and small data.	Full context: memory map, registers, file handles, PCB , etc.
Overhead	Small overhead.	Large overhead.
Performance	Very fast, suitable for real-time tasks.	Slower compared to threads.
When used?	When one program is doing multiple tasks internally.	When computer runs multiple programs.
Example	YouTube app: switching between video thread, audio thread, comment loading thread .	Switching between Chrome, VS Code, Spotify running on your computer.